DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling

Xiaoxing Wang, Zhirui Lian, Jiale Lin, Chao Xue, and Junchi Yan, *Senior Member, IEEE*

***Abstract*—Despite its popularity as a one-shot Neural Architecture Search (NAS) approach, the applicability of differentiable architecture search (DARTS) on complex vision tasks is still limited by the high computation and memory costs incurred by the over-parameterized supernet. We propose a new architecture search method called EasyNAS, whose memory and computational efficiency is achieved via our devised operator merging technique which shares and merges the weights of candidate convolution operations into a single convolution, and a dynamic channel refinement strategy. We also introduce a configurable search space-to-supernet conversion tool, leveraging the concept of atomic search components, to enable its application from classification to more complex vision tasks: detection and semantic segmentation. In classification, EasyNAS achieves state-of-the-art performance on the NAS-Bench-201 benchmark, attaining an impressive 76.2% accuracy on ImageNet. For detection, it achieves a mean average precision (mAP) of 40.1 with 120 frames per second (FPS) on MS-COCO test-dev. Additionally, we transfer the discovered architecture to the rotation detection task, where EasyNAS achieves a remarkable 77.05 mAP_{50} on the DOTA-v1.0 test set, using only 21.1 M parameters. In semantic segmentation, it achieves a competitive mean intersection over union (mIoU) of 72.6% at 173 FPS on Cityscape, after searching for only 0.7 GPU-day.**

***Index Terms*—Channel reduction, image classification, neural architecture search, object detection, operation merge, semantic segmentation, supernet construction.**

I. Introduction

HANDCRAFTED neural architectures designed by experts have achieved promising performance across computer vision tasks [1], [2], but they require extensive trials and errors along with expert experience. To address this limitation, Neural Architecture Search (NAS) has emerged as a promising field, aiming to automatically discover neural architectures with little human interference. In recent years, NAS has been intensively explored, leading to significant advancements in architecture design. Reinforcement learning (RL)-based approaches [3], [4] have been proposed to generate neural architectures by training an agent. Genetic algorithms [5], [6] have also been employed, leveraging crossover and mutation operations to generate new architectures. Unlike the aforementioned methods that regard the search process as a black box, DARTS [7] (namely differentiable architecture search) and its variants [8], [9] introduce architecture parameters to indicate the importance of candidates in the search space. By training a supernet through SGD algorithm, DARTS solves a bi-level optimization problem and can discover efficient neural architectures within a few GPU days. A few works [10], [11], [12] show the unsatisfactory robustness[1] of DARTS, also known as performance collapse issue. Specifically, skip connection will gradually dominate other operations, making performance of the discovered architectures increases at first and then declines along the search process [12]. RobustDARTS [10] attributes such an issue to the overfitting of architectural parameters and propose to regularize the training of supernet weights to improve the robustness of DARTS. With similar motivation, DARTS+ [13] introduces early stop strategy and SDARTS [11] proposes a perturbation-based regularization technique. DARTS- [14] adds an auxiliary skip edge to each cell, which can also be regarded as a regularization on skip-connection operation. ZARTS [15] introduces to solve the bi-level optimization of NAS problem by the zero-order optimizer to alleviate the performance collapse issue.

However, despite the successes of NAS in classification tasks, its application to more challenging tasks such as detection and segmentation has been limited. These tasks require larger GPU memory and computational resources during the search process, and they involve more complex search spaces. Previous NAS methods tailored to detection [16] and segmentation [17], [18] tasks often suffer from high search costs or inflexible task-dependent implementations. To improve the usability of NAS for vision especially in a spirit of 'DIY', we on one hand should

Manuscript received 29 January 2023; revised 24 June 2023; accepted 20 July 2023. Date of publication 24 July 2023; date of current version 3 October 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2020AAA0107600, in part by the National Natural Science Foundation of China under Grant 62222607, and in part by Shanghai Municipal Science and Technology Major Project under Grant 2021SHZDZX0102. Recommended for acceptance by N. Vasconcelos. *(Correspondence author: Junchi Yan.)*

Xiaoxing Wang, Zhirui Lian, and Jiale Lin are with the Department of Computer Science and Engineering, MoE Key Lab of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: figure1_wxx@sjtu.edu.cn; lzr0705@sjtu.edu.cn; linjiale@sjtu.edu.cn).

Chao Xue is with the School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: xch@bupt.edu.cn).

Junchi Yan is with Shanghai AI Lab, Shanghai 100876, China (e-mail: yanjunchi@sjtu.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TPAMI.2023.3298296, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2023.3298296

[1]Robustness of a NAS method describes whether it can stably discover networks with high performance along the search process.

0162-8828 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

improve the search efficiency especially for its GPU memory cost; and on the other hand, we should develop a reusable and configurable NAS tool to support different vision tasks which is mainly considered at the implementation level. We analyze the difficulties for achieving such goals in the following:

1) It requires large amounts of memory and computation costs to direct search for complex vision tasks, such as detection and segmentation. On the one hand, the datasets for these vision tasks are much larger with higher input resolutions. On the other hand, the models are much deeper and more complicated than those for classification. Such an issue restricts the application of one-shot NAS [19], DARTS [7], and their variants since it is intractable to build and train an over-parameterized supernet.

Works [20], [21] propose to reduce the memory and computation costs for classification tasks by sampling and activating a subset of the supernet at each iteration. However, applying them to complex vision tasks can be challenging since sampling results in dynamic network structures, affecting the supernet training and decreasing the convergence speed. Other NAS methods aim at detection or segmentation but still requires vast search costs. Some detection NAS works propose to search the architecture of either backbone [16], [22] or feature pyramid network (FPN) [23], [24], [25], but requires hundreds of GPU-days. AutoDeepLab [18] and FasterSeg [17] apply DARTS to segmentation tasks but require large GPU memory to train a supernet. Therefore, it is a standing demand for an efficient architecture search method with high speed and low memory cost, especially with as light user efforts as possible.

2) It demands a unified and configurable conversion tool from custom search spaces to construct supernets. Specifically, the network architectures for various tasks significantly vary, e.g., detection models contain backbone and feature pyramid networks, while segmentation models contain encoder and decoder modules. The prior works usually focus on a single vision task and adopt an inflexible task-dependent implementation, and few have been explored across various vision tasks. Development of such a conversion tool is nontrivial due to the demand for flexibility and user-friendly. On the one hand, it should support arbitrary search spaces for various vision tasks; On the other hand, users can easily design a specific search space with no need to code for the complicated supernet builder.

To this end, this paper proposes EasyNAS that can DIY neural architectures for various vision tasks. Our contributions can be summarized as follows[2]:

1) A novel convolution operator merge scheme for memory and time efficient architecture search. This work proposes an operator merging technique to speed up the search procedure and reduce the GPU memory requirement by merging the convolutions with different kernel sizes and dilation rates into one. Compared to sampling-based DARTS methods [20], [21], our method maintains a static supernet architecture which can alleviate the convergence difficulty of training supernet and thus is more suitable for complex computer vision tasks. Experiments on classification and detection tasks verify the better convergence property[3] of EasyNAS than sampling-based methods (Fig. 3).

2) An efficient channel reduction search scheme for compact architecture discovery via dynamic channel sampling. Few DARTS-based methods [27] have been explored to search channel numbers which are usually not differentiable. To this end, we sample channel numbers at each iteration through the Gumbel reparameterization technique and introduce a dynamic channel refinement strategy to reduce the computation cost. Unlike the prior works that have redundant computations due to zero padding on channels, our method activates the sampled channels at each iteration and has no redundant computations.

3) A configurable search space-to-supernet conversion tool applicable to different vision tasks. Different vision tasks often involve distinct neural architectures and thus require custom search spaces and supernets. To this end, we propose an atomic search component, which serves as the basic brick to flexibly stack a supernet. We also develop the tool so that the users can design their search space through a simple configuration file (yaml format in our implementation) and a supernet can be automatically converted. In particular, we show the supernets built through our component in Fig. 6 (classification), Fig. 7 (detection), and Fig. 8 (segmentation). To our best knowledge, this is the first work for a pipeline reporting its search results on the above vision tasks. Based on the proposed supernet builder, operator merging technique, and dynamic channel refinement strategy, we derive a memory-efficient and computing-efficient neural architecture search method, named EasyNAS for easy deployment of NAS.

4) State-of-the-art performance across datasets and vision tasks, including a new NAS task – rotation detection. We apply EasyNAS to three classic vision tasks and derive EasyNAS(cls) for classification, EasyNAS(det) for object detection, and EasyNAS(seg) for semantic segmentation. Specifically, in the classification task, EasyNAS(cls) can discover efficient architectures with less memory cost and searching time on the search space of DARTS [7] and NAS-Bench-201 [28]. In the detection task, EasyNAS(det) achieves outstanding performance on classic horizontal and rotation detection tasks: 40.1 mAP with 120 FPS on MS-COCO and 77.05 mAP_{50} on DOTA. In the semantic segmentation task, EasyNAS(seg) achieves 72.6% mIoU with 28 G FLOPs and 174 FPS after searching for 0.7 GPU-day, outperforming FasterSeg with more than twice the acceleration. To the best of our knowledge, our method EasyNAS is the first framework supporting the direct search for various complex CV

[2]Part of the work was published in the two conference works: [8] which applies the merge mechanism to classification, and [26] which designs a sophisticated search space for detection and applies the merge technique. The journal version aims to unify these methods under a single framework and makes the extensions and improvements: 1) to make it applicable to different vision tasks with user specified search space, we design and implement a reusable and configurable conversion tool from the given search space to supernet based on our proposed concept of atomic search component; 2) extensive additional experiments including i) classification on ImageNet as well as robustness analysis which is an emerging issue in recent NAS; ii) more detection results using four different detection heads beyond YOLO; iii) brand new results on segmentation which also verifies our method's general applicability; iv) the paper was rewritten with more comprehensive discussions on our approach and related works.

[3]Better convergence property lies in two aspects: faster convergence speed of architecture parameters α and better performance of the supernet. The former can be indicated by the entropy curve of α, and the latter can be indicated by the accuracy curve of the supernet.

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

tasks in a few GPU-days. Source code will be made publicly available at: https://github.com/SJTU-Thinklab/EasyNAS.

II. Related Work

A. Neural Architecture Search

1) Classic NAS for Image Classification: Several approaches to classical NAS include reinforcement learning, evolutionary algorithms, and traditional methods (such as grid search and hand-designed heuristics). Reinforcement learning approaches to NAS, such as NASNet [3] involve training a neural controller network to generate new architectures. The controller is trained using reward signals based on the performance of the generated architectures. Evolutionary algorithms [5], [6] simulate the principle of natural selection to evolve a population of neural network architectures. The architectures are trained and evaluated, and good architectures are preserved to produce the next generation. This process is repeated until an optimal architecture is found. Classical NAS methods rely on a combination of human expertise and pre-defined rules [29] to search through the space of possible architectures and identify the best one for a given task. Besides the above time-consuming methods, one-shot NAS [19] can reduce the search cost to a few GPU-days. DARTS [7] regards NAS as a bi-level optimization problem and proposes to solve it by a differentiable method. Our paper also adopts the differentiable method due to its efficacy and high efficiency.

2) NAS With Extension for Object Detection: Existing detection frameworks usually consist of four modules: backbone, feature fusion neck, region proposal network (in two-stage detectors), and detection head. For real-time detection, [2], [30], [31] design efficient architectures for the four modules, especially for the YOLO series methods. There are also emerging manually-designed detectors for rotated object detection whereby different loss functions are carefully devised, ranging from regression [32], [33], [34] to classification [35], [36] models. Unlike the above works requiring massive trials and expert experience to design CNNs, some works aim to automatically search architectures for object detection. Recent NAS methods can be briefly categorized into three streams: 1) Search backbone architecture and fix FPN, e.g., DetNAS [16] and SP-NAS [22]. 2) Search FPN architecture and fix backbone, e.g. NAS-FPN [23], Auto-FPN [24], and NAS-FCOS [25]. 3) Jointly search backbone and FPN, e.g., SM-NAS [37], Hit-Detector [38] and our method. Unlike SM-NAS and Hit-Detector which require vast search costs, our work introduces operator merging and dynamic channel refinement techniques that can significantly reduce the GPU memory requirement during the search process. Specifically, our EasyNAS can search under a more extensive search space on the MS-COCO dataset directly on a single V100 GPU in 1.4 days. Moreover, unlike many NAS detection methods [16], [37], [38], [39] that need to pre-train supernets on the ImageNet, our EasyNAS trains the supernet from scratch on the MS-COCO during the searching process, demonstrating its outstanding convergence ability.

3) NAS With Extension for Semantic Segmentation: Many manually-designed architectures [40], [41] have achieved remarkable performance on the semantic segmentation task by introducing dilated convolution and atrous spatial pyramid pooling (ASPP) modules. Recent works attempt to utilize NAS to search for more efficient architectures for the segmentation task. AutoDeeplab [18] applies DARTS to speed up the search and designs a search space consisting of various operations and complex connections. FasterSeg [17] also applies DARTS and introduces zoomed operation to reduce the computation cost. However, though DARTS shows its high efficiency in speed, it requires larger amounts of GPU memory to train a complex supernet. In contrast, this work applies the proposed operator merging technique and can achieve competitive and even better performance in less search time and GPU-memory.

4) NAS for Other Vision Problems and Beyond: Besides the aforementioned three mainstream vision tasks, NAS has also been recently adopted in other vision tasks e.g. imaging [42], pose estimation [43], 3-D point cloud recognition [44]. In addition, other tasks beyond vision like speech recognition [45] and variational quantum algorithms [46]. When the supernet involves convolution operators our method in general can be (partly) applied.

The above works are mostly designated separately for the specific task and there lacks an (especially explicitly) unified and reusable tool to cover different tasks especially for vision, as classification, detection and segmentation indeed share similar and common building blocks and basic principles. This motivates us to develop a principled and efficient paradigm for NAS in vision.

B. Towards NAS Across Datasets/Tasks/Devices

Apart from task-specific NAS which is the focus of most existing works on NAS as discussed above, some efforts have also been made towards more general settings from different perspectives. In [47], the warm start for NAS from one dataset to another is considered and the searched architecture is expected to bear good performance on a new dataset. Some works [4], [7] attempt to transfer the architectures between similar datasets, but they mainly focus on the classification task. NAT [48] also proposes to adapt pre-trained supernets across datasets in classification based on transfer learning, and attempts to transfer the supernet to semantic segmentation. However, it relies on the evolutionary algorithm and requires large amounts of search costs. In addition, NAS across devices has also been studied in [49], which trains a once-for-all (OFA) network that efficiently supports diverse architectural settings by decoupling training and search. DONN [50] introduces a regularization item of latency to search for architectures with various sizes for different devices.

Compared to the above works, it is worth noting that our approach indeed is not aimed at finding an architecture that can be reused across tasks, datasets, or devices by certain means. While the purpose is to develop a cost-effective and general methodology that can be engineering-readily applied (at the code level which is clean and reusable) to various vision tasks. Though at the first glance, the vanilla DARTS [7] manages to achieve this, yet in practice, it can be challenging for its high memory demand especially for detection and segmentation, as discussed in Section I.

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

C. Memory/Computing-Efficient DARTS

As aforementioned, the one-shot DARTS framework has been a popular search paradigm in NAS, while it requires high memory to train the supernet. Therefore, a few works [20], [21], [51] are dedicated to reducing the memory and computation cost for DARTS. [20] samples $1/K$ channels for each operation in the one-shot model to reduce the memory cost and accumulate the architecture importance of various iterations to stabilize the optimization. [21], [51] utilize a reparameterization strategy with Gumbel-softmax to sample only a subset of operations in the supernet at every iteration, which can significantly reduce the memory cost since only the weights of selected operations contributes to the output. Another line of works [52], [53], [54] refers to the pruning technique [55] and propose to gradually prune weak operations from the supernet so that the memory and computation cost will decrease along with the pruning process. ASAP [54] proposes an annealing schedule to decay the temperature of softmax and utilizes a threshold policy to prune weak operations according to the value of architecture parameters. SGAS [53] further defines selection certainty and stability to measure the saliency of each edge, and greedily prunes all weak operations on one edge. In contrast, this paper introduces to reduce memory by merging convolutions into one. Unlike the sampling-based and pruning-based methods that have a dynamic supernet structure during the search process, our operator merging technique maintains a static supernet structure and can have a better convergence property.

III. Methodology

We first present the overview of our approach in Section III-A, as comprised of three key parts: i) convolution operator merging as described in Section III-B; ii) channel reduction search as introduced in Section III-C; iii) a reusable and configurable tooling scheme for supernet construction in Section III-D.

A. Approach Overview

One-shot NAS methods [7], [19] build a supernet containing all candidate operations and connections. Neural network architectures can be regarded as directed acyclic graphs (DAG), where nodes denote feature maps and edges denote connections between layers. For node j, we utilize $\mathcal{G}_j$ to denote the candidate set of its predecessors. For each predecessor node of j, $\forall g \in \mathcal{G}_j$, we utilize e_{gj} to denote the edge from node g to node j, and $e_{gj} = \{f^o_{gj}\}_{o\in\mathcal{O}}$ is a set of candidate operations on the edge where $\mathcal{O}$ is the search space including convolution, pooling, and identity operations. We then introduce architecture parameters α^o_{gj} and β_{gj} to represent the importance of candidate operation f^o_{gj} and connection from node g to j. The feature map at node j in the supernet, denoted as z_j, can be computed as

$$z_j = \sum_{g\in\mathcal{G}_j} \hat{\beta}_{gj} \sum_{o\in\mathcal{O}} \hat{\alpha}^o_{gj} f^o_{ij}(z_g) \tag{1}$$

where $\hat{\alpha}^o_{gj} = \frac{\exp(\alpha^o_{gj})}{\sum_{o'\in\mathcal{O}} \exp(\alpha^{o'}_{gj})}, \hat{\beta}_{gj} = \frac{\exp(\beta gj)}{\sum_{g'\in\mathcal{G}_j} \exp(\beta g'j)}$ are normalized architecture parameters through softmax.

Training such a supernet requires large GPU memory since (1) computes $|\mathcal{O}| \times |\mathcal{G}_j|$ operations, and all the latent feature maps should be saved for backpropagation. Additionally, since the discovered architecture is only a small sub-network of the supernet, those weights in the supernet are usually redundant, which will also result in an over-fitting issue. In this paper, we make the following specific efforts to achieve an efficient and general framework, and the overview of our approach is displayed in Fig. 1.

In Section III-B, we propose an operator merging technique to reduce the GPU memory requirement and computation cost, which is applicable to general vision tasks.

Moreover, in complex vision tasks, e.g., detection and segmentation, the number of model parameters can be much larger than classification, which increases the difficulty of NAS. However, few works have explored to search for them. Hence, in Section III-C, we consider searching for the channel reduction rate and propose a dynamic channel refinement strategy based on Gumbel reparameterization. After sampling channel reduction rates through the Gumbel-argmax technique, we refine the weights of operations to fit the changeable channel numbers, which can reduce the computation cost of supernet forward.

In Section III-D, to concretely make our search method applicable to different tasks and datasets, we introduce a configurable conversion tool from custom search spaces to supernets constructed by stacking atomic search components. Our implementation is flexible enough to support various search spaces for classic vision tasks, including classification, detection, and segmentation.

B. Memory-Efficient Search via Convolution Operator Merging

A search space consists of parameterless and parametric operations in one-shot based NAS methods [7], [19]. The former includes zero operation, identity operation, and pooling operation, and the latter contains multiple types of convolutions with different kernel sizes and dilation rates representing various receptive fields. Based on the homogeneity and linearity of convolution, we can simplify the weighted sum of convolutions and reduce the memory and computation costs. Before that, we would like first to state the following propositions. We omit the subscript (gj) for conciseness in the following.

Proposition 1: The convolution operation with kernel $\mathbf{W}$ is equivalent to a set of convolution operations whose kernel $\tilde{\mathbf{W}}$ is derived from $\mathbf{W}$ by padding $2n(n \geq 0)$ zeros around.

Proposition 2: The dilated convolution operation with kernel size k_1 and dilation rate r is equivalent to a normal convolution operation with kernel size $k_2 = r(k_1 - 1) + 1$. In the kernel of the normal convolution, there exist r zeros between two non-zero weights.

These two propositions can be derived according to the computation of convolution. Detailed proofs can be found in the appendix, available online. Propositions 1 and 2 show that we can substitute each candidate convolution in the search space with an equivalent convolution with a larger kernel size and a mask. Masks for various convolutions are shown in Fig. 2,

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

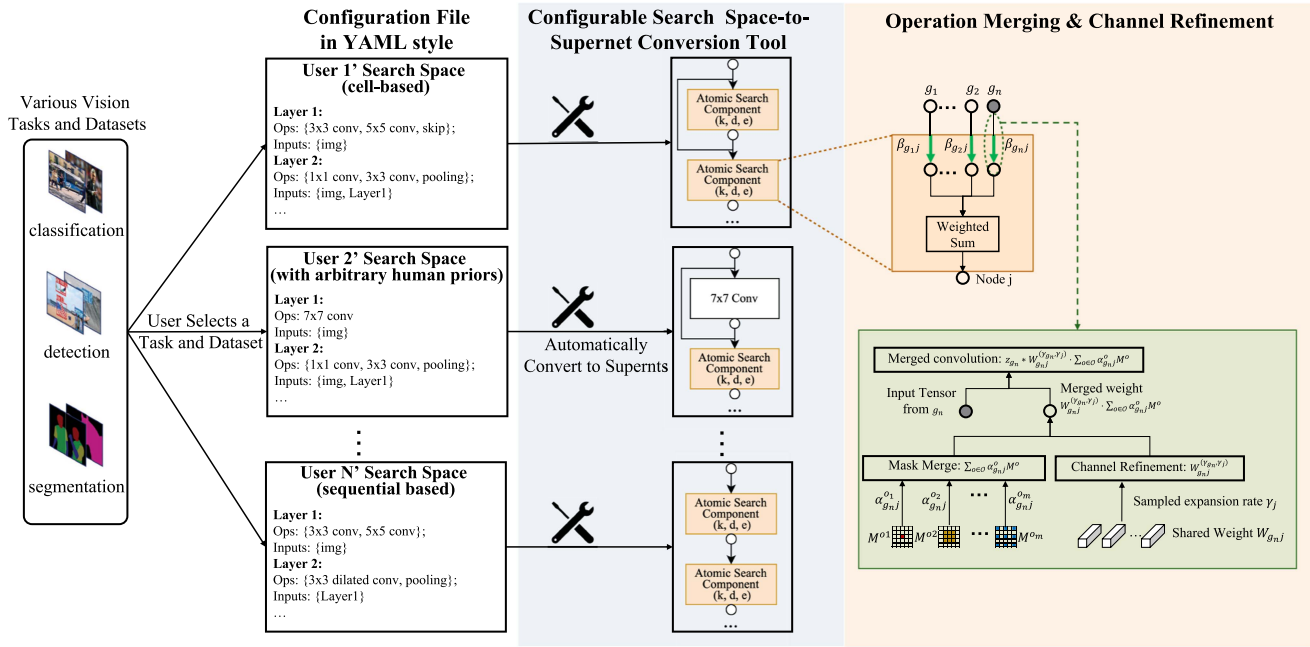
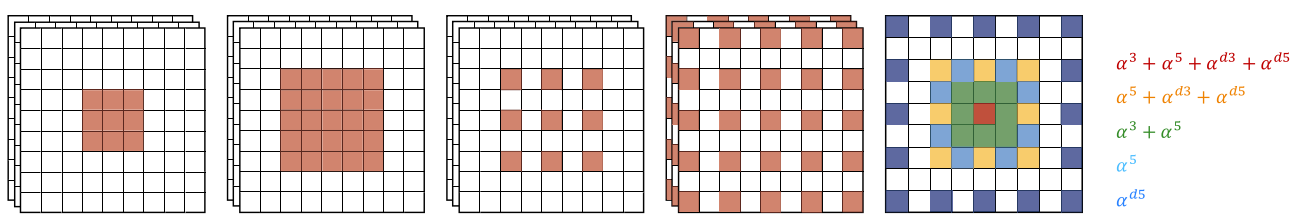


Fig. 1. Overview of the proposed EasyNAS which aims to provide a general, easy-to-use tool for NAS in vision. Users can design search spaces for arbitrary vision tasks and datasets and provide a configuration file in the YAML style. Our search space-to-supernet conversion tool can automatically construct the corresponding supernets by stacking atomic search components. Notice that our tool supports various kinds of search spaces, including cell-based micro spaces and sequential macro spaces. Architectures can be searched in a few GPU-days (commodity GPU) by training the supernet through our operator merging technique and channel refinement strategy. Specifically, in each atomic search component, kernels of candidate operations can be extracted from unified shared weights so that multiple convolutions can be compounded into one convolution. The shared weights are dynamically refined before each iteration according to the sampled reduction rate.



(a) Mask for 3×3 convolution (b) Mask for 5×5 convolution (c) Mask for 3×3 dilated convolution (d) Mask for 5×5 dilated convolution (e) mask matrix $\tilde{M}$

Fig. 2. (a)-(d) denote the equivalent mask M^o indicating the receptive field for different convolutions, where the red block denotes non-zero weights. (e) is the weighted average mask $\tilde{M}$ according to the architecture parameters α. Different colors refer to the corresponding coefficients.

where the white block indicates zero, and the red block indicates one. Therefore, we can then simplify the weighted sum of convolutions as follows:

$$\sum_{o \in \mathcal{O}^p} \hat{\alpha}^o \left[z * \mathbf{W}^o \right] = z * \left[\sum_{o \in \mathcal{O}^p} \hat{\alpha}^o \tilde{\mathbf{W}}^o \right] = z * \tilde{\mathbf{W}}, \tag{2}$$

where $\mathcal{O}^p$ is the set of candidate convolutions in the search space, and $\tilde{\mathbf{W}}^o$ is the equivalent weights with large kernel size for the convolution o based on Propositions 1 and 2.

(2) shows that various convolutions can be merged into one. We can obtain the weighted average of the convolutions by conducting convolution once. Thus the computation cost and GPU memory can be significantly reduced.

However, the number of parameters in the supernet is unchanged, and the model with redundant parameters still suffers the overfitting problem from the redundant weights. To this end, we introduce an operator merging technique by sharing the weights among different convolutions. In the following, we first introduce the operator merging technique for normal convolutions. Then we expand it to separable convolutions.

1) Operator Merging for Normal Convolutions: To reduce the number of parameters in the supernet, we first define a tensor $\mathbf{W}$ with the maximal kernel size for each edge, from which the weights of all other convolutions can be derived. We then use mask matrices $\mathbf{M}^o, \forall o \in \mathcal{O}^p$ to denote the receptive field of different convolutions, as shown in Fig. 2. Correspondingly, the weights of convolution o can be derived by $\tilde{\mathbf{W}}^o = \mathbf{M}^o \cdot \mathbf{W}$. Based on Proposition 1, the convolution with weights $\tilde{\mathbf{W}}^o$ is equivalent to the original convolution o. Therefore, the weights of the merged convolution $\tilde{\mathbf{W}}$ in (2) can be obtained by (3)

$$\tilde{\mathbf{W}} = \sum_{o \in \mathcal{O}^p} \hat{\alpha}^o \tilde{\mathbf{W}}^o = \left[\sum_{o \in \mathcal{O}^p} \hat{\alpha}^o \mathbf{M}^o \right] \cdot \mathbf{W} = \tilde{\mathbf{M}} \cdot \mathbf{W} \tag{3}$$

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

The matrix $\tilde{\mathbf{M}} = \sum \hat{\alpha}^o \mathbf{M}^o$ denotes the importance of different receptive fields. By sharing the weights among convolutions, the operator merging technique can reduce the amount of computation and the number of parameters. Thus, the computation of z_j in (1) can be simplified as

$$z_j = \sum_{g \in \mathcal{G}_j} \hat{\beta}_{gj} \left[\sum_{o \in O \backslash \mathcal{O}^p} \hat{\alpha}^o_{gj} f^o_{ij}(z_g) + z_g * \mathbf{W}_{gj} \cdot \sum_{o \in \mathcal{O}^p} \hat{\alpha}^o_{gj} \mathbf{M}^o \right] \tag{4}$$

where $\mathbf{W}_{gj}$ denotes the weight shared among convolutions on edge e_{gj}, $\mathcal{O}^p$ is the set of convolutions, and $\mathcal{O} \backslash \mathcal{O}^p$ is the set of parameterless operations.

2) Operator Merging for Separable Convolutions: Many approaches [7], [19] involve separable convolutions in the search space due to their high efficiency. To apply the operator merging technique to separable convolutions, we have the following proposition, which exposes the relationship between separable convolution and normal convolution. We omit the subscript (gj) for conciseness.

Proposition 3: In the condition of no activation and normalization layers between the depth-wise and point-wise convolution, the separable convolution is equivalent to a normal convolution, whose weight tensor is the Hadamard Product of the weight tensors of depth-wise and point-wise convolution.

Detailed proof can be found in the appendix, available online. Separable convolutions in many search spaces [4], [7], [19] satisfy the above condition. Based on the associative law of convolution, the weights of the equivalent normal convolution $\mathbf{W}_s$ can be obtained by (5), where $\mathbf{W}^o_d \in \mathbb{R}^{C_i \times 1 \times k \times k}$, $\mathbf{W}^o_p \in \mathbb{R}^{C_o \times C_i \times 1 \times 1}$ are the weights of the depth-wise and the point-wise convolution, $\mathbf{W}^{o\top}_d \in \mathbb{R}^{1 \times C_i \times k \times k}$ is the transposed version of the $\mathbf{W}^o_d$ along the first two dimensions, and '$\cdot$' denotes Hadamard product

$$\begin{aligned} \mathbf{W}^o_s &= \mathbf{W}^o_d * \mathbf{W}^o_p \\ &= \mathbf{W}^{o\top}_d \cdot \mathbf{W}^o_p \end{aligned} \tag{5}$$

By sharing the weights of depth-wise convolution, The weights of the merged convolution can be computed as follows based on the linearity of the Hadamard product

$$\begin{aligned} \tilde{\mathbf{W}} &= \sum_{o \in \mathcal{O}^p} \alpha^o \mathbf{M}^o \cdot \mathbf{W}^\top_d \cdot \mathbf{W}^o_p \\ &= \mathbf{W}^\top_d \cdot \sum_{o \in \mathcal{O}^p} \alpha^o \mathbf{W}^o \cdot \mathbf{W}^o_p \end{aligned} \tag{6}$$

We can further share the weights of point-wise convolution, denoted as $\mathbf{W}_p$, and obtain the weights of the merged convolution as follows:

$$\begin{aligned} \tilde{\mathbf{W}} &= \mathbf{W}^\top_d \cdot \sum_{o \in \mathcal{O}^p} \alpha^o \mathbf{M}^o \cdot \mathbf{W}_p \\ &= \left[\tilde{\mathbf{M}} \cdot \mathbf{W}_d \right]^\top \cdot \mathbf{W}_p \\ &= \mathbf{W}'_d * \mathbf{W}_p \end{aligned} \tag{7}$$

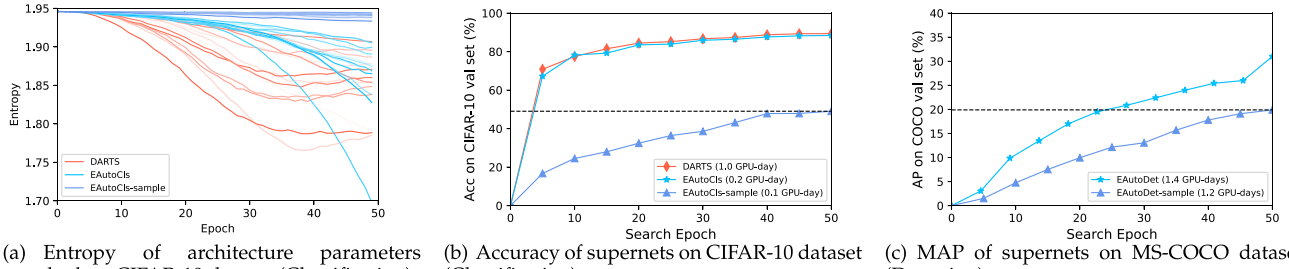
Consequently, the weighted average of separable convolutions can be merged into a single separable convolution whose depth-wise weight is $\mathbf{W}'_d$ and point-wise weight is $\mathbf{W}_p$. The search space of DARTS contains separable convolutions, and the effectiveness of our operator merging technique is evaluated on it, which is detailed in Section IV-A.

3) Analysis on Operator Merging Technique: Better convergence property: Apart from our operator merging technique, sample-based methods [20], [21] are also popular in reducing the memory and computation cost for classification. However, they involve a dynamic network structure due to sampling sub-networks of the supernet at every iteration, affecting the supernet training. Though such an issue can be alleviated on classification tasks by increasing the training epochs, it is unsuitable for complex vision tasks e.g. object detection as training one epoch requires a much higher cost. In contrast, our operator merging technique reduces GPU memory and computation cost by compounding multiple kernels into one. Meanwhile, it holds a stable network structure without interfering with supernet training. Our experiments also show the better convergence property of our method: 1) faster convergence speed of the architecture parameters α and 2) better training performance of the supernet. For the former, we plot entropy curves of α in Fig. 3(a). We observe that the entropy of α in our EAutoCls and DARTS descends much faster than the sampling-based method, showing the faster convergence speed of α. Regarding the latter one, we train the supernet on classification and object detection tasks and plot the performance (Accuracy/mAP) curve of the supernet in Fig. 3(b) and (c). We observe that our EasyNAS converges better and faster than the sample-based method.

Robustness analysis on the performance collapse issue: Recent works [10], [11], [12] show the unsatisfactory robustness of DARTS, which is also known as the performance collapse issue. Specifically, the skip connection operations will gradually dominate other candidate operations, resulting in initially improved performance of the discovered architectures that later declines during the search process. The works [10], [13] attribute this issue to the overfitting of architectural parameters α. Empirical evidence from RobustDARTS [10] demonstrates that α falls into a sharp minimum[4], leading to overfitting during the search process. DARTS+ [13] further shows that once overfitting occurs, the α values associated with skip connection operations gradually increase, causing them to dominate the normal cell. To address this, they propose to improve the robustness of DARTS by alleviating overfitting through L2 regularization, dropout technique, or early stop strategy. In this work, we introduce an operator merging technique that significantly reduces the number of supernet parameters, which can also alleviate the overfitting issue and improve the robustness of DARTS. To verify the aforementioned analysis and evaluate its robustness, we follow [12] by training the supernet for 200 epochs, obtaining the discovered architectures at $\{10, 15, 20, 25, 50, 75, 100, 125, 150, 175, 200\}$ epoch, and retrain those networks for 600 epochs from scratch to evaluate their ultimate performance. Fig. 4 compares the

[4]A sharp minimum lies in a sharper region of the loss landscape, as defined in RobustDARTS [10].

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.



(a) Entropy of architecture parameters searched on CIFAR-10 dataset (Classification) (b) Accuracy of supernets on CIFAR-10 dataset (Classification) (c) MAP of supernets on MS-COCO dataset (Detection)

Fig. 3. (a) We plot the entropy curve of architecture parameters α searched on the classification task. Lower entropy values imply that architecture parameters are trained to convergence so that important operations are distinguishable. The opacity of color indicates the entropy curve of α on different edges. The larger the opacity, the deeper the edges in a normal cell. (b)(c) We plot the performance trend of supernets trained for 50 epochs. The search cost is given in the legend. EasyNAS-sample denotes to search by sample-based method [21]. The horizontal dash line is the final performance (Acc/mAP) of the sample-based method. The performance trend of DARTS on classification task is nearly coincident with EasyNAS(cls), demonstrating the good convergence property of our method. In contrast, the sample-based method performs rather worse.

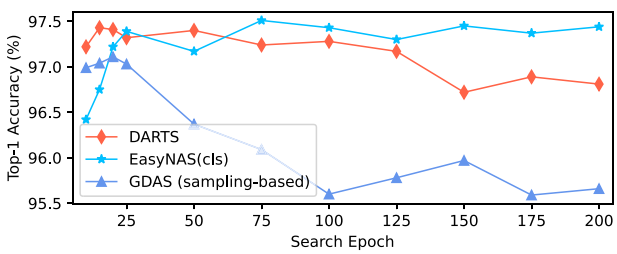


Fig. 4. Trends of ultimate accuracy of the discovered architecture in the search process of DARTS, EasyNAS(cls), and GDAS which is a popular sampling-based differentiable NAS method. The top-1 accuracy of each network is obtained by training for 600 epochs from scratch.

accuracy curve of EasyNAS, DARTS, and the popular sampling-based differentiable NAS methods – GDAS. We observe that networks discovered by EasyNAS(cls) perform consistently well (around 97.40% accuracy) while the performance of networks discovered by DARTS and GDAS drops.

Difference from the Prior Works: Unlike the prior NAS works [8] that focus on classification tasks, our method aims to search detection models that are more complex and requires more computation resources. Our operator merging technique can significantly reduce memory and computation costs, making it possible to discover an effective architecture in a few GPU-days. Apart from NAS works, RepVGG [56] is also related to our approach, which introduces a re-param strategy to merge skip-connection, 3×3 and 1×1 convolutions for a plain inference-time model. However, the motivation of RepVGG is to stabilize the training of VGG, and the merge process is applied after the training stage. In contrast, our operator merging technique is applied during the searching process and aims to reduce the memory and computation costs.

C. Channel Reduction Search via Dynamic Sampling and Refinement

Layer channels are essential hyperparameters for neural network architectures affecting the model size and FLOPs. A few works [27] also show that manually-designed channel numbers contain redundancy, and they make an effort to reduce it by searching for channel numbers via the Gumbel reparameterization technique. However, though they sample and activate a subset of channels at each iteration, they pad the channel dimension via zeros to the maximal channel numbers. Such a padding method maintains a unified channel dimension but leads to useless computations in the search process. In contrast, we introduce a dynamic channel refinement technique based on Gumbel reparameterization technique by refining the operation weights W dynamically to fit the changeable channel numbers. Our method removes redundant computations and can further speed up the search process.

1) Gumbel-Softmax to Sample Reduction Rate: Suppose $\mathcal{R}$ is a set of reduction rate candidates. We introduce $\boldsymbol{\gamma}_j = [\gamma_j^r]_{r \in \mathcal{R}}$ to represent the importance of each reduction rate for the merged convolutions on edges $e_{gj}, \forall g \in \mathcal{G}_j$, that take j as the end node. Hence, the reduction rate for node j can be sampled by the Gumbel-argmax technique

$$\mathbf{R}_j = \text{one_hot}\left[\arg\max_{r \in \mathcal{R}}(\log \tilde{\gamma}_j^r + \xi_j^r)\right], \tag{8}$$

where $\tilde{\gamma}_j^r = \frac{\exp(\gamma_j^r)}{\sum_{r' \in \mathcal{R}} \exp(\gamma j^{r'})}$ is the normalized weights through softmax, and ξ_j^r is a random variable sampled from Gumbel$(0, 1)$ distribution. To make $\mathbf{R}_j$ differentiable w.r.t. $\boldsymbol{\gamma}_j$, we adopt Gumbel-softmax to relax the sampled vector as follows, where τ is the gradually decayed temperature

$$\tilde{\mathbf{R}}_j = \frac{\exp\left[(\log \tilde{\boldsymbol{\gamma}}_j + \xi_j)/\tau\right]}{\sum_{r \in \mathcal{R}} \exp\left[(\log \tilde{\gamma}_j^r + \xi_j^r)/\tau\right]}, \tag{9}$$

where $\xi_j = [\xi_j^r]_{r \in \mathcal{R}}$ is a vector of Gumbel variables. We adopt the one-hot vector $\mathbf{R}$ ((8)) to activate one candidate reduction rate in forward pass and utilize the relaxed vector $\tilde{\mathbf{R}}$ ((9)) to obtain gradients for $\boldsymbol{\gamma}$ in back-propagation. It is a popular reparameterization technique that has been widely-use in many recent works [21], [27].

2) Dynamic Channel Refinement: Suppose the sampled reduction rate r_j for node j with the maximal output channel number C_j, and then the actual output channel becomes $r_j C_j$. The operation weights $W_{gj}, \forall g \in \mathcal{G}_j$ can be refined by preserving the first $r_j C_j$ filters. Besides, it affects the convolutions' input channel that takes node j as one of the predecessors. Therefore,

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

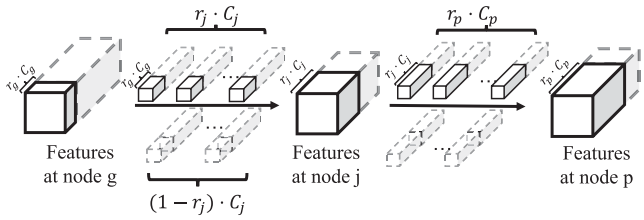


Fig. 5. Refinement for convolution weights W on adjacent layers in our dynamic channel refinement technique. r indicates the sampled reduction rate. Dotted blocks in light gray indicate the inactivated channels.

TABLE I
SEARCH COST OF PADDING ZERO TECHNIQUE IN [27] AND OUR DYNAMIC CHANNEL REFINEMENT STRATEGY

Method	Baseline	Zero padding ([27])	Dynamic Channel Refinement (ours)
Cost (GPU-hours)	36.6	41.9	33.1

We search architectures for detection tasks on MS-COCO. The supernet is built based on EasyNas(det)-YOLOv5s (see fig. 7). Each supernet is trained for 50 epochs. 'Baseline' represents not searching for channel numbers.

we dynamically refine channels of weights in adjacent convolutions, as shown in Fig. 5. Generally, before each iteration, we sample reduction rates for each node j and refine the output channels of convolutions on edge $e_{gj}, \forall g \in \mathcal{G}_j$ that take j as the end node, and the input channels of convolutions on edge $e_{jk}, \forall k : j \in \mathcal{G}_k$ that take j as the input node.

Compared to the zero-padding technique in [27], our dynamic channel refinement strategy has a lower search cost, as shown in Table I. Suppose $\mathcal{R}$ is the set of candidate reduction rates. The sampled candidate's weights at a specific iteration are $\hat{\gamma}$. Then the FLOPs ratio of FBNet to our method is $1/\mathbb{E}_{\hat{\gamma}}[\mathcal{R}]$. Specifically, in our settings, $\mathcal{R} = \{0.5, 0.75, 1.0\}$ and $\hat{\gamma}$ is uniformly initialized as $\{1/3, 1/3, 1/3\}$. Then the FLOPs ratio of FBNet to our method is $4/3$, in accordance with the results in Table I.

Transformation for Concatenation Layers: The channel number for each node alters dynamically during search, which brings difficulty in refining weights for convolutions after a concatenation layer. Specifically, suppose two features with reduction rates and base channels (r_1, C_1) and (r_2, C_2) are concatenated. Convolution is applied after the concatenation layer, then the activated input channels of weight are separated ($0 \sim r_1 C_1$ and $C_1 \sim C_1 + r_2 C_2$), making it hard to extract activated weights. To address this issue, we introduce the following proposition.

Proposition 4: The output of a concatenation layer followed by a convolution layer is equivalent to the sum of separate convolutions on the inputs.

Detailed proof can be found in the appendix, available online, which can be derived from the linearity of convolutions. Consequently, we can transfer concatenation layers to sum layers without any loss to alleviate the issue of separate activated channels.

D. The Configurable Search Space-to-Supernet Conversion Tool Via Atomic Search Component Stacking

This work aims to explore a general search method for various computer vision tasks, which requires custom search spaces and distinct supernets. By analyzing the search spaces and supernets for classification [7], [8], object detection [26], and semantic segmentation [17], we observe that most of the supernets are constructed by stacking atomic components with two searching functions: i) candidate operations; ii) connections (input layer). We propose to define a flexible atomic search component that can support various custom search spaces. Specifically, it can be regarded as a directed acyclic graph with an arbitrary number of input nodes $\mathcal{G}_j$ and one output node j, and serves to compute (1). It also supports searching for the output channel numbers. In contrast, a relevant effort is presented in AtomNAS [57], which proposes an atom block as the basic component of their search space. However, the topology of their atom block is fixed consisting of two convolutions and a channel-wise operation which cannot fulfill the general search space building purpose in our work – they only search for image classification.

We will introduce the proposed atomic search component whose input layers and candidate operation can be flexibly designed by custom. Moreover, we show that various supernets for classification, detection, and segmentation can be built by stacking such an atomic search component.

1) Atomic Search Component: The architecture is displayed in Fig. 1, where the candidate operations for each candidate input layer are independent, consistent with the definition of search spaces in other NAS methods [4], [7], [8], [21], [26]. Suppose node j is the output of an atomic search component without loss of generality. The search space can be defined as

Algorithm 1: EasyNAS for General Architecture Search (Applicable to Classification, Detection, Segmentation, etc.).

Input:
1) A supernet constructed by stacking atomic search component with search space $\{\mathcal{G}, \mathcal{O}, \mathcal{R}\}$;
2) The initialized sharing weights $\mathbf{W}$ to merge convolutions on each edge;
3) The mask matrix for each operation $\mathbf{M}^o, o \in O$;
Hyperparameters:
1) The maximal epoch T to train the supernet weights and architecture parameters;
Output:
1) The optimal architecture parameters α, β, γ;

for $t = 1$ **to** T **do**
  Get normalized architecture parameters $\{\hat{\alpha}, \hat{\beta}, \hat{\gamma}\}$ through softmax;
  Sample reduction rate based on $\hat{\gamma}$ for each node through (8);
  Apply dynamic channel refinement to convolution weights based on Fig. 5;
  Supernet forward by computing the atomic search component in turn by (10);
  Update architecture parameters $\{\alpha, \beta, \gamma\}$ and supernet weights W alternately through SGD.
end for
return the optimal architecture parameters $\{\alpha, \beta, \gamma\}$

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

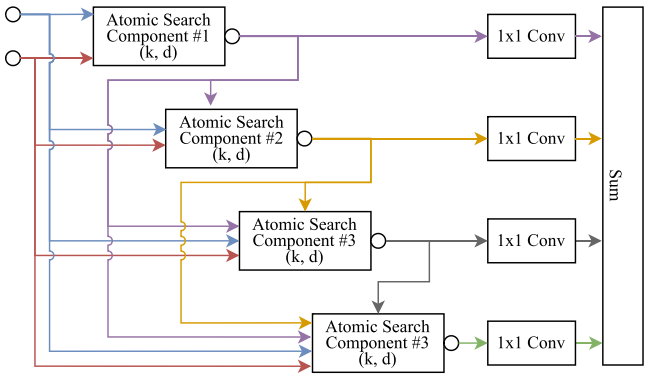


Fig. 6. Architecture of the cell for classification task based on the search space of DARTS [7]. The architecture of the atomic search component is illustrated in Fig. 1. Lines in the same color are from the same source node. The candidate operation set $\mathcal{O}$ contains eight operations in each atomic search component.

$\{\mathcal{G}_j, \mathcal{O}_j, \mathcal{R}_j\}$, where $\mathcal{G}_j$ is a set of candidate input layers, $\mathcal{O}_j$ is a set of candidate operations, an $\mathcal{R}_j$ is a set of candidate reduction rates. We define architecture parameters α, β, γ to represent the importance of each candidate.

Together with the operator merging technique and dynamic channel refinement, the output of the atomic search component z_j can be computed as (10). Note that we take normal convolutions as an example and omit the parameterless operations for conciseness

$$z_j = \sum_{g \in \mathcal{G}_j} \hat{\beta}_{gj} \left[z_g * \mathbf{W}_{gj}^{(r_g, r_j)} \cdot \sum_{o \in \mathcal{O}_j^p} \hat{\alpha}_{gj}^o \mathbf{M}^o \right] \tag{10}$$

where $\mathbf{M}^o$ denotes the mask matrix for operation o shown in Fig. 2, $\mathbf{W}_{gj}^{(r_g, r_j)}$ denotes the shared weight tensor of all candidate convolutions where r_g and r_j are the sampled reduction rates of input and output channel through Gumbel reparameterization technique. Based on the PyTorch platform, we implement the atomic search component by defining a python class based on the torch.nn.module.

2) Supernet Construction: A supernet can be built by stacking the above atomic search component. We implement two ways to construct supernets: one is by defining a supernet class based on the torch.nn.module in PyTorch, and the other is by defining a supernet architecture through a yaml file, and the code will automatically build the supernet class in the first way according to the yaml file. Our implementation can cover most of the definition of search spaces, and Figs. 6, 7, 8 illustrates the classic supernets built by our atomic search component for classification, detection, and segmentation. Our EasyNAS algorithm is listed in Algorithm 1.

IV. Experiments

To verify the effectiveness of EasyNAS, we conduct experiments on three classic computer vision tasks and derive EasyNAS(cls) for classification, EasyNAS(det) for object detection, and EasyNAS(seg) for semantic segmentation.

For each task, we divide the training set of each dataset into two parts to obtain architecture parameters and network weights. After alternately optimizing architecture parameters and network weights for 50 epochs through an SGD optimizer, the final architecture is derived by preserving the connection, operation, and reduction rate according to the architecture parameter. Table II compares the memory costs of our EasyNAS and DARTS on three tasks, showing that our method has similar memory efficiency as the sampling-based method (GDAS [21]).

TABLE II
GPU Memory Cost of EasyNAS and DARTS on Three Tasks

Method	Classification	Object Detection	Segmentation
DARTS [7]	9.9 G (↓0%)	16.5 G (↓0%)	12.4 G (↓0%)
GDAS [21]	3.1 G (↓68%)	10.6 G (↓36%)	7.1 G (↓43%)
EasyNAS	4.6 G (↓54%)	11.0 G (↓33%)	7.3 G (↓41%)

The search spaces for the three tasks are described in Section IV-A2, IV-B2, and IV-C2. The relative memory reduction against darts is in the bracket.

The methods are implemented by PyTorch 1.7.1 platform. The experiments on classification tasks are conducted on NVIDIA 1080 Ti GPU, and the experiments on detection and segmentation tasks are conducted on NVIDIA V100 GPU. The results of other methods are quoted from the original papers unless otherwise stated.

A. Architecture Search for Classification

1) Protocol: We derive the image classification version of EasyNAS and term it as EasyNAS(cls). We search and evaluate on CIFAR-10 dataset and compare it with the related works. Then we transfer the discovered architecture to the ImageNet dataset to test its transferability.

Dataset: CIFAR 10 [58] consists of ten classes, including lane, car, bird, cat, deer, dog, frog, horse, ships, and trucks. CIFAR100 [58] has 100 classes. The training set has 500 images per category, and the validation set has 100 images per category. The size of each image is 32×32. ImageNet [59] contains 14,197,122 images, 841 synset and 1000 classes. The training set has 1,281,167 images with labels. Each category contains about 1300 images. The validation set has 50,000 images, 50 per category. It has been one of the most popular benchmarks to evaluate the performance of CNN models.

Experimental Settings: We set the training settings similar to DARTS. In the search phase, we train the supernet stacked by 8 cells for 50 epochs and alternately optimize the architecture parameters and the network weights. SGD optimizer with a momentum of 0.9 and an initial learning rate of 0.025 is used to optimize the network weights. Adam optimizer with initial learning rate 10^{-4} is used to optimize the architecture parameters. After the search process, the discovered architecture is trained for 600 epochs from scratch to test its ultimate performance.

2) Search Space (Standard): The efficiency and stability of EasyNAS(cls) is evaluated on two cell search spaces: the micro search space of DARTS [7] and the search space in NAS-Bench-201 [28].

In the former search space, a supernet is built by stacking normal and reduced cells. Each cell contains four intermediate nodes and two input nodes. Each node is connected with all the previous nodes and the input nodes. The architecture of

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

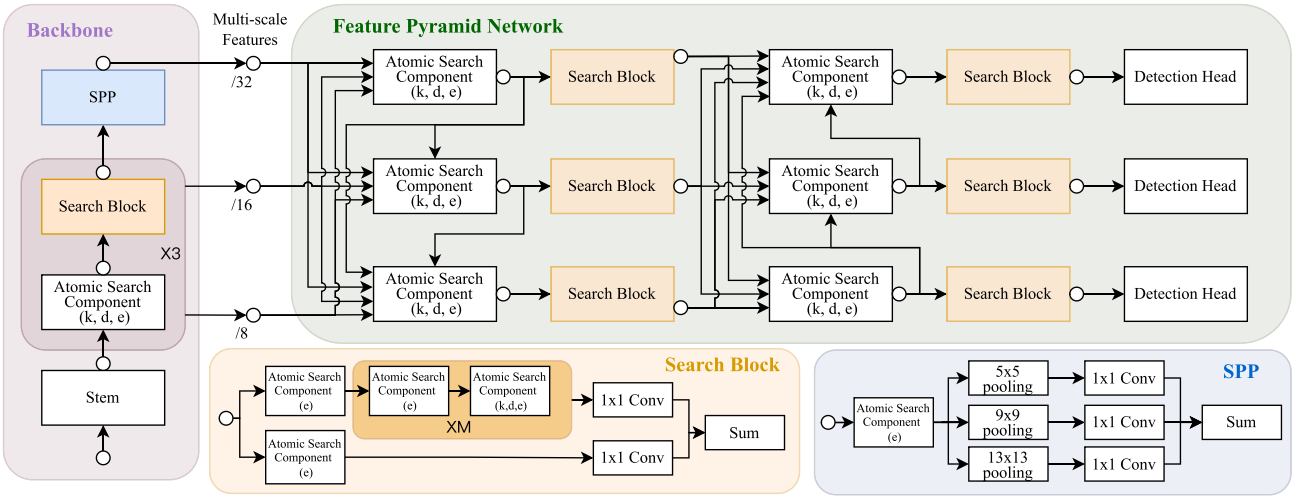


Fig. 7. Architecture of supernet, containing all candidate operations and connections in the search space. The architecture of the atomic search component is given in Fig. 1. In the backbone, search block, and SPP module, parentheses under 'Atomic Search Component' indicate hyper-parameters to search: kernel size k, dilation ratio d, and reduction rate of output channels e.

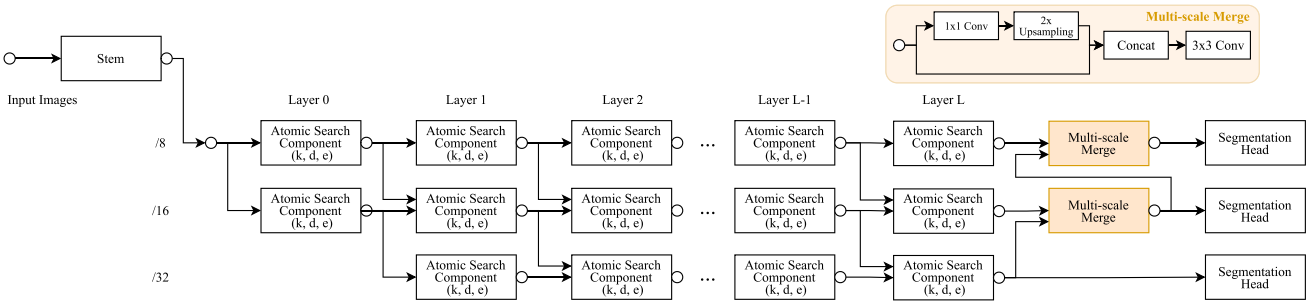


Fig. 8. Architecture of supernet based on the search space of FasterSeg [17]. The atomic search component is illustrated in Fig. 1. The parentheses under 'Atomic Search Component' indicate hyper-parameters to search: kernel size k, dilation ratio d, and reduction rate of output channels e. The 'Multi-scale Merge' block has the same architecture as that in FasterSeg.

the cell built by our atomic search component is illustrated in Fig. 6. In each atomic search component, the candidate operation set $\mathcal{O}$ contains eight operations, including zero, identity, max pooling, average pooling, 3×3 separable convolution, 5×5 separable convolution, 3×3 dilated separable convolution, and 5×5 dilated separable convolution. It should be note that in the search space of DARTS, each candidate separable convolution is doubled to improve the performance. In our settings, we utilize single separable convolution, similar to ENAS [4].

The latter search space is designed by NAS-Bench-201, which only searches architectures for normal cells and maintains the reduced cell as the bottleneck structure of ResNet [1]. The set $\mathcal{O}$ contains 5 basic operations: none, skip connect, average pooling, 1×1 convolution, and 3×3 convolution. NAS-Bench-201 has evaluated the validation and test accuracy of all the possible architectures in the search space so that we can focus on the searching process and directly index for the information of the architecture discovered by our approach.

3) Results by the Search Space of DARTS on CIFAR and ImageNet: In the evaluation phase, a large network stacked by 20 cells, where the 6th and 13th cells are the reduced cell, is trained from scratch with batch size 96. Note that the convolutions in the final network is single separable convolutions in our operation set. Similar to DARTS [7], we use the cutout augmentation strategy and auxiliary classifier in the evaluation phase. We search for the architecture based on EasyNAS(cls), and optimize the network weights and architecture parameters iteratively based on 1st and 2nd DARTS [7]. Comparison with other NAS methods is given in Table III. EasyNAS(cls) (1st order) discovers an efficient architecture in 0.2 GPU-day, achieving 97.35% averaged accuracy on CIFAR10. EasyNAS(cls) (2nd order) can discover an even better architecture in 0.6 GPU-day, achieving 97.45% averaged accuracy. Compared to DARTS and SNAS, EasyNAS(cls) achieves better performance with fewer parameters and less search time.

We also train the discovered architecture of EasyNAS(cls) (2nd order) on the ImageNet dataset. Notice that in the search space of DARTS, each candidate separable convolution is doubled to improve the performance. We utilize single separable convolution in our settings, similar to ENAS [4]. For a fair comparison with prior works, we report the performance of the architecture under the two settings in Table IV. Specifically, the

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON WITH STATE-OF-THE-ART IMAGE CLASSIFIERS ON CIFAR10 (LOWER ERROR RATE IS BETTER)

Classification Method	Top-1 Err. (%) ↓	Params (M) ↓	Cost (GPU-days) ↓
NASNet-A [3]	2.65	3.3	1800
AmoebaNet-B [6]	2.55±0.05	2.8	3150
Hierarchical Evo [5]	3.75±0.12	15.7	300
PNAS [29]	3.41±0.09	3.2	225
ENAS [4]	2.89	4.6	0.5
DARTS (1st order) [7]	3.00±0.14	3.3	0.4
DARTS (2nd order) [7]	2.76±0.09	3.4	1.0
SNAS (moderate) [60]	2.85±0.02	2.8	1.5
ProxylessNAS [61]	2.08	5.7	4.0
P-DARTS [52]	2.50	3.4	0.3
PC-DARTS (1st order) [20]	2.57±0.07	3.6	0.1
BayseNAS [62]	2.81±0.04	3.4	0.2
EasyNAS(cls) (1st order)	2.65±0.08	2.9	0.2
EasyNAS(cls) (2nd order)	2.55±0.07	3.0	0.6

Similar to darts, the search cost for EasyNAS does not include the selection or final evaluation cost. The performance values of compared methods are excerpted from the raw paper. We evaluate EasyNAS(cls) three times under different random seeds and report the averaged accuracy.

TABLE IV
PERFORMANCE ON IMAGENET IN DARTS'S SEARCH SPACE [7] BY TWO ARCHITECTURES

Classification Method	FLOPs (M) ↓	Params (M) ↓	Top-1 Err. (%) ↓	Cost (GPU-days) ↓
AmoebaNet-A [6]	555	5.1	25.5	3150
NASNet-A [63]	564	5.3	26.0	1800
PNAS [29]	588	5.1	25.8	225
DARTS (2nd) [7]	574	4.7	26.7	1.0
P-DARTS [52]	557	4.9	24.4	0.3
PC-DARTS [20]	586	5.3	25.1	0.1
SNAS [60]	522	4.3	27.3	1.5
GDAS [21]	581	5.3	26.0	0.2
ProxylessNAS† [61]	465	7.1	24.9	8.3
EasyNAS(cls)-singleSep	485	4.3	26.0	0.2
EasyNAS(cls)	696	6.0	23.8	0.2

'Singlesep' denotes that the separable convolution in the search space is single, similar to ENAS [4]; Otherwise, the separable convolution in the search space is double, similar to darts [7].

discovered architecture outperforms most priors with a similar number of parameters.

Due to GPU memory limits, DARTS [7] has to search the architecture of normal and reduced cells on a shallow supernet (8 cells) but evaluate the ultimate performance on a deep network (20 cells). However, a performance gap in the depth transfer in one-shot NAS exists. Our approach reduce the memory cost in the search phase, making it possible to directly search on a deeper supernet (20 cells). Hyper-parameters are set similarly to DARTS. Results are presented in Table V, showing that directly searching on a deep supernet can obtain a better architecture than transferring from a shallow neural architecture.

4) Results by the Search Space of NAS-Bench-201 on CIFAR and ImageNet: NAS-Bench-201 provides detailed information on all possible architectures belonging to the specific search space, including the accuracy and latency on three datasets: CIFAR10 [58], CIFAR100 [58], and ImageNet-16-120 [28]. ImageNet-16-120 contains 120 different classes in ImageNet dataset [59], and all images are reshaped to 16×16. The training settings and split protocol can be found in [28]. We can directly index for the accuracy of the network searched by EasyNAS(cls) instead of training from scratch. We search for the architecture based on DARTS and EasyNAS(cls) three times with different random seeds. The results are in Table VI , showing that our method outperforms other methods on most datasets.

TABLE V
RESULTS ON CIFAR10 OF THE NETWORKS WITH 20 CELLS WHOSE STRUCTURE IS SEARCHED BY EASYNAS(CLS) ON THE SUPERNET STACKED BY 8/20 CELLS

Number of Cells		Acc	Params
Search	Evaluation	(%)	(M)
8 (1st order)	20	97.35±0.08	2.9
8 (2nd order)	20	97.45±0.07	3.0
20 (1st order)	20	97.47±0.09	2.9

TABLE VI
COMPARISON WITH STATE-OF-THE-ART IMAGE CLASSIFIERS ON NAS-BENCH-201

Architecture	CIFAR10		CIFAR100		ImageNet-16-120	
	valid	test	valid	test	valid	test
RSPS [64]	84.16	87.66	59.00	58.33	31.56	31.14
GDAS [21]	90.00	93.51	71.14	70.61	41.70	41.84
SETN [65]	82.25	86.19	56.86	56.87	32.54	31.90
ENAS [4]	39.77	54.30	15.03	15.61	16.43	16.32
DARTS (wd=0.01) [7]	67.29	87.36	59.90	59.80	36.06	35.58
DARTS (wd=0.02) [7]	64.44	86.74	57.67	57.38	31.09	30.78
EasyNAS(cls) (wd=0.01)	76.41	91.89	69.28	69.44	42.38	41.87
EasyNAS(cls) (wd=0.02)	84.75	**94.36**	**73.49**	**73.51**	**46.37**	**46.34**
Optimal	91.61	94.37	73.49	73.51	46.77	47.31

The accuracy of other methods, except DARTS, is directly cited from the raw paper. The optimal is provided by the NAS-Bench-201, which indicates the best architecture in the search space. We evaluate darts and EasyNAS(cls) three times under different random seeds and report the averaged accuracy. 'wd' indicates weight decay.

B. Architecture Search for Object Detection

1) Protocol: We apply EasyNAS to the object detection task and derive the variant of our approach as termed by EasyNAS(det). We conduct experiments on two popular detection tasks: classic detection and rotation detection. The former is typical in many detection contests to locate common objects. The latter has been widely used in aerial images aiming to locate the ground object instances with an oriented bounding box (OBB). To verify EasyNAS(cls), we first search on a large and general dataset for classic detection tasks and then evaluate the performance of our discovered architectures on both classic and rotation detection tasks.

Dataset: For the classic detection task, we adopt MS-COCO 2017 benchmark with 80 common object categories, which contains 118 K training images, 5 K validation images, and 41 K test images in 80 common object categories. For the rotation detection task, we adopt the DOTA-v1.0 benchmark, one of the most popular aerial detection benchmarks. The fully annotated DOTA benchmark contains 15 object categories: Plane (PL), Baseball diamond (BD), Bridge (BR), Ground field track (GTF), Small vehicle (SV), Large vehicle (LV), Ship (SH), Tennis court

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
COMPARISON WITH PRIOR WORKS ON THE COCO TEST-DEV

Object Detection Method	Resolution	FPS	#Params (M)	mAP (%)	AP_{50} (%)	AP_{75} (%)	AP_S (%)	AP_M (%)	AP_L (%)	Search Cost
YOLOv4 [30]	416	96	-	41.2	62.8	44.3	20.4	44.4	56.0	-
YOLOv5s† [66]	640	113	7.3	36.9	56.0	40.0	19.9	41.1	46.0	-
YOLOv7-tiny [67]	640	182	6.2	38.6	56.8	41.5	21.2	42.7	52.3	-
EfficientDet-D0 [39]	512	98	3.9	33.8	52.2	35.8	12.0	38.3	51.2	-
NAS-FPN [23]	640	24	60.3	39.9	-	-	-	-	-	333*
NAS-FCOS@128 [25]	1333×800	-	27.8	37.9	-	-	-	-	-	28
SpineNet-49S [68]	640	-	11.9	39.5	59.3	43.1	20.9	42.2	54.3	-
SM-NAS:E2 [37]	800×600	25	-	40.0	58.2	43.4	21.1	42.4	51.7	187
EasyNAS(det)-YOLOv5s (ours)	**640**	**120**	**9.1**	**40.1**	**58.7**	**43.5**	**21.7**	**43.8**	**50.5**	**1.4**
EasyNAS(det)-YOLOv7-tiny (ours)	**640**	**159**	**7.0**	**38.9**	**57.2**	**41.2**	**19.9**	**42.0**	**54.8**	**1.9**
YOLOv3 + ASFF [69]	416	54	-	40.6	60.6	45.1	20.3	44.2	54.1	-
YOLOv4 [30]	512	83	-	43.0	64.9	46.5	24.3	46.1	55.2	-
YOLOv4-csp [31]	512	80†	43	46.2	64.8	50.2	24.6	50.4	61.9	-
YOLOv5m† [66]	640	88	21.4	43.9	62.5	47.6	25.1	48.1	54.9	-
EfficientDet-D1 [39]	640	74	6.6	39.6	58.6	42.3	17.9	44.3	56.0	-
DetNAS [16]	1333×800	-	-	42.0	63.9	45.8	24.9	45.1	56.8	44
NAS-FPN [23]	1024	13	60.3	44.2	-	-	-	-	-	333*
Auto-FPN [24]	800	-	32.6	40.5	61.5	43.8	25.6	44.9	51.0	16
NAS-FCOS@256 [25]	1333×800	-	57.3	43.0	-	-	-	-	-	28
SpineNet-49 [68]	640	-	28.5	42.8	62.3	46.1	23.7	45.2	57.3	-
SM-NAS:E3 [37]	800×600	20	-	42.8	61.2	46.5	23.5	45.5	55.6	187
Hit-Detector [38]	1200×800	-	27.1	41.4	62.4	45.9	25.2	45.0	54.1	-
OPA-FPN@64 [70]	1333×800	22	29.5	41.9	-	-	-	-	-	4
EasyNAS(det)-YOLOv5m (ours)	**640**	**70**	**28.1**	**45.2**	**63.5**	**49.1**	**25.7**	**49.1**	**57.3**	**2.2**
YOLOv3 + ASFF [69]	608	46	-	42.4	63.0	47.4	25.5	45.7	52.3	-
YOLOv4 [30]	608	62	-	43.5	65.7	47.3	26.7	46.7	53.3	-
YOLOv4-csp [31]	640	65†	53	47.5	66.2	51.7	28.2	51.2	59.8	-
YOLOv5l† [66]	640	59	47.1	46.8	65.4	50.9	27.7	51.0	58.5	-
YOLOv7 [67]	640	109	36.9	50.7	69.1	55.1	35.0	55.6	65.6	-
EfficientDet-D2 [39]	768	57	8.1	43.0	62.3	46.2	22.5	47.0	58.4	-
SPNet(BNB) [22]	1333×800	10	-	45.6	64.3	49.6	28.4	48.4	60.1	156‡
SM-NAS:E5 [37]	1333×800	9	-	45.9	64.6	48.6	27.1	49.0	58.0	187
OPA-FPN@160 [70]	1333×800	13	60.6	47.0	-	-	-	-	-	4
EasyNAS(det)-YOLOv4-csp (ours)	**640**	**55**	**49.8**	**47.8**	**66.1**	**51.9**	**28.6**	**51.5**	**60.1**	**4.2**
EasyNAS(det)-YOLOv5l (ours)	**640**	**59**	**34.4**	**47.9**	**66.3**	**52.0**	**28.3**	**52.0**	**59.9**	**4.5**
EasyNAS(det)-YOLOv7 (ours)	**640**	**100**	**39.9**	**50.5**	**68.8**	**55.0**	**33.7**	**55.5**	**65.6**	**4.1**
YOLOv3 + ASFF [69]	800	29	-	43.9	64.1	49.2	27.0	46.6	53.4	-
YOLOv5x† [66]	640	43	87.8	49.1	67.5	53.6	30.2	53.4	61.4	-
EfficientDet-D3 [39]	896	35	12	45.8	65.0	49.3	26.6	49.4	59.8	-
SPNet(XB) [22]	1333×800	6	-	47.4	65.7	51.9	29.6	51.0	60.4	156‡
EasyNAS(det)-YOLOv5x (ours)	**640**	**41**	**86.0**	**49.2**	**67.5**	**53.6**	**30.4**	**53.4**	**61.5**	**22**

FPS for YOLOv5 and our method are calculated on a single V100 GPU, and results for other methods are directly obtained from their papers. Different blocks indicate models with various inference speeds and prediction performance. '†': The results are obtained by our experiments. '–': the value is not provided by the original paper. '*': the unit of search cost is TPU-days, while the unit of other methods is GPU-days. '‡': SPNet [22] shows the search cost on VOC is 26 GPU-days, and is six times lower than that on COCO.

TABLE VIII
PERFORMANCE OF THE DISCOVERED ARCHITECTURE WITH VARIOUS CLASSIC DETECTION HEADS

Detector	ResNet50-FPN		EasyNAS(det) (ours)	
	#Param (M)	mAP (%)	#Param (M)	mAP (%)
RetinaNet	38.0	36.5	27.2 (↓10.8)	38.1 (↑1.6)
FCOS	32.3	38.7	24.4 (↓7.9)	41.5 (↑2.8)
Faster-RCNN	41.8	37.4	33.7 (↓8.1)	39.3 (↑1.9)
Mask-RCNN	44.4	38.2	36.4 (↓8.0)	39.8 (↑1.6)

The results of ResNet50-FPN are obtained from mmdetection (https://github.Com/open-mmlab/mmdetection). Models are initialized by pre-trained weights and trained for 12 epochs. Improvements are in brackets.

TABLE IX
JOINT SEARCH VS. INDEPENDENT SEARCH FOR BACKBONE AND FPN ON MS-COCO VALIDATION SET

Detection Method		s-level (small)		m-level (medium)	
Backbone	**FPN**	**mAP**(%)	**Δ**(%)	**mAP**(%)	**Δ**(%)
default	default	36.9	+0.0	44.0	+0.0
searched	default	37.4	+0.5	44.6	+0.6
default	*searched*	38.9	+2.0	45.0	+1.0
searched	***searched***	**40.1**	**+3.2**	**45.5**	**+1.5**

Results show the superiority of joint search against independent search.

(TC), Basketball court (BC), Storage tank (ST), Soccer-ball field (SBF), Roundabout (RA), Harbor (HA), Swimming pool (SP), and Helicopter (HC).

Experimental Settings: This work adopts the YOLO detector and searches the architecture of the backbone and FPN module. The discovered architectures are trained from scratch for 300 epochs by an SGD optimizer. We directly utilize the hyper-parameters provided by YOLOv5 for a fair comparison. To fairly compare the speed (FPS) with YOLO methods, we convert the trained models to the style of YOLOv4 [30] and evaluate the FPS on the Darknet platform [70], which is implemented in C and CUDA. We also transfer the discovered architectures of backbone and FPN to Faster-RCNN [71], RetinaNet [2], FCOS [72],

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

TABLE X
ABLATION STUDY ABOUT THE DYNAMIC CHANNEL SAMPLING AND REFINEMENT STRATEGY

Search for Reduced Channel Number	Search Cost (GPU-hour)	#Params (M)	mAP (%)
×	36.6	10.9	39.8
✓	33.1	9.1	40.1

The supernet is built based on EasyNAS(det)-YOLOv5s, as shown in Fig. 7. Each supernet is trained for 50 epochs, and each discovered architecture is trained for 300 Epochs.

and Mask-RCNN [73] detectors to verify the generalization among different types of detector heads. Besides, we evaluate the generalization of the discovered architectures by transferring them to the rotation detection task. Specifically, we train models on the training set of DOTA-v1.0 from scratch for 300 epochs and evaluate on the validation and test sets. Notice that we train and test on a single input scale, unlike previous works [32], [33] that adopt multi-scale training techniques and random rotation augmentation.

2) Search Space (Self-Designed): Detection CNN models usually contain two sub-modules: one is the backbone to extract features in multi-scale, and the other is FPN to fuse the multi-scale features. For the backbone, we design the search space based on YOLOv5 [65], YOLOv4-csp [31], and YOLOv7 [66] to discover optimal kernel sizes, dilation rates, and channel reduction rates. For the FPN module, we design the search space to discover optimal connections.

Search Space for Backbone: In the following, we take the search space designed based on YOLOv5 as an example and introduce it in detail. We propose to search operation types and channel numbers for down-sampling operators and search blocks. The supernet built by stacking the atomic search components is shown in Fig. 7. *1)* Down-sampling operators have four candidates: {1x1 conv, 3x3 conv, 5x5 conv, 3x3 dilated conv} and three choices of reduction rate for output channel: $\{0.5, 0.75, 1.0\}$; *2)* Search block has two 1x1 convolutions with two choices of reduction rates: $\{0.75, 1.0\}$ and several stacked units. Each unit consists of two convolutions with three choices of reduction rate: $\{0.5, 0.75, 1.0\}$, and the second convolution has three candidates: {3x3 conv, 5x5 conv, 3x3 dilated conv}. Architectures of different layers are independently searched.

Search Space for Feature Pyramid Network: Supernet for the FPN module is shown in Fig. 7, which enables to search connections of features in three spatial scales, operation types and channel numbers of each connection, and the structure of search blocks. *1)* Nodes indicate feature maps and connect with all their predecessors in the supernet. After the search stage, only two predecessors will be selected for each node; *2)* In each atomic search component, the candidate operation set contains {1x1 conv, 3x3 conv, 5x5 conv, 3x3 dilated conv} with three possible reduction rates for the output channel: $\{0.5,0.75,1.0\}$; *3)* The search blocks, whose architectures are also searched, are concatenated at the end of each atomic search component to independently extract multi-scale features.

3) Results on Horizontal Detection Benchmark: COCO: Table VII reports the performance of our methods and compares them with other state-of-the-art works on the COCO test-dev dataset. Different blocks indicate models with various inference speeds and prediction performance. We observe that our discovered models (EasyNAS(det)) achieve the best performance. Specifically, EasyNAS(det)-YOLOv5s achieves 40.1 mAP with 120 FPS, outperforming EfficientDet-D0 by 6.3% mAP with similar inference speed. Compared to manually-designed detectors (YOLO series), our method has competitive and even better performance. EasyNAS(det)-YOLOv5m achieves 45.2% mAP with 69.9 FPS, surpassing YOLOv4 by 2.2% mAP. Moreover, we compare to YOLOv4-csp [31] by inheriting its backbone CD-53 and searching FPN architecture ('EasyNAS(det)-YOLOv4-csp' in Table VII). The discovered architecture outperforms YOLOv4-csp by 0.3% mAP with fewer parameters. Moreover, we notice that the superiority of NAS mainly affects light-weighted models: Our discovered s-level model outperforms YOLOv5 by more than 3 Additionally, we transfer the discovered architecture of EasyNAS(det)-YOLOv5m to adapt to Faster-RCNN [71], RetinaNet [2], FCOS [72], and Mask-RCNN [73] detector heads, and compare it with ResNet50 since they have a similar parameter number. Results are reported in Table VIII, showing that our method can also benefit other detectors, and verifying the generalization of the discovered architecture among different types of detector heads.

4) Discussion on the Advantage of Joint Search for Backbone and FPN: Many prior works [16], [23], [24] requires large GPU memory to train a supernet and thus have to search on a limited search space, i.e., only search the architecture for backbone or FPN module. In contrast, the operator merging technique can significantly reduce the GPU memory requirement, making our method possible to search on a large joint space of backbone and FPN. To show the advantage of searching on a joint space, we compare the performance of joint and independent search for backbone and FPN in Table IX, where 'default' indicates that we directly adopt the architecture of YOLOv5, and 'searched' indicates that we search for the architectures. We observe that: 1) Joint search achieves the best performance, showing the effectiveness of our algorithm and the necessity of joint search for detection models. 2) The search performance is much more sensitive to the architecture of FPN than the backbone module.

5) Ablation Study of Reduced Channel Number Search: To test the effectiveness of channel search, we conduct ablation studies and report the results in Table X. Specifically, we build a supernet based on EasyNAS(det)-YOLOv5s and train it for 50 epochs w/ and w/o the search for reduced channel numbers. The discovered architecture is trained for 300 epochs from scratch to test the ultimate performance. We observe that our dynamic channel sampling and refinement strategy can reduce the search cost by nearly 10% and discover a more light-weighted architecture with competitive and even slightly better mAP performance.

6) Results on Rotation Detection Benchmark: DOTA: We utilize Circular Smooth Label (CSL) technique [36] to obtain robust angular prediction through classification without suffering boundary conditions. The baseline adopts RetinaNet [2] detection framework with ResNet152 [1] backbone and FPN [83]

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
COMPARISON ON THE TEST SET OF ORIENTED BOUNDING BOXES (OBB) TASK IN DOTA-V1.0 FOR ROTATION DETECTION

Rotation Detection Method	PL	BD	BR	GTF	SV	LV	SH	TC	BC	ST	SBF	RA	HA	SP	HC	mAP_{50}
ReDet(ReR50) [32]⋆	88.79	82.64	53.97	74.00	78.13	84.06	88.04	**90.89**	87.78	85.75	61.76	60.39	75.96	68.07	63.39	76.25
CenterMap(R101) [76]⋆	89.83	84.41	**54.60**	70.25	77.66	78.32	87.19	90.66	84.89	85.27	56.46	69.23	74.13	71.56	66.06	76.03
CSL(R152) [36]⋆ †	**90.13**	84.43	54.57	68.13	77.32	72.98	85.94	90.74	85.95	86.36	63.42	65.82	74.06	73.67	70.08	76.24
O^2-DNet(H104) [77]	89.31	82.14	47.33	61.21	71.32	74.03	78.62	90.76	82.23	81.36	60.93	60.17	58.21	66.98	61.03	71.04
DAL(R101) [78]	88.61	79.69	46.27	70.37	65.89	76.10	78.53	90.84	79.98	78.41	58.71	62.02	69.23	71.32	60.65	71.78
P-RSDet(R101) [79]	88.58	77.83	50.44	69.29	71.10	75.79	78.66	90.88	80.10	81.71	57.92	63.03	66.30	69.77	63.13	72.30
BBAVectors(R101) [80]	88.35	79.96	50.69	62.18	78.43	78.98	87.94	90.85	83.58	84.35	54.13	60.24	65.22	64.28	55.70	72.32
DRN(H104) [81]†	89.71	82.34	47.22	64.10	76.22	74.43	85.84	90.57	86.18	84.89	57.65	61.93	69.30	69.63	58.48	73.23
DCL(R152) [35]†	89.10	84.13	50.15	73.57	71.48	58.13	78.00	**90.89**	86.64	86.78	67.97	67.25	65.63	74.06	67.05	74.06
PolarDet(R101) [82]†	89.65	**87.07**	48.14	70.97	78.53	80.34	87.45	90.76	85.63	86.87	61.64	**70.32**	71.92	73.09	67.15	76.64
GWD(R152) [33]†	86.96	83.88	54.36	**77.53**	74.41	68.48	80.34	86.62	83.41	85.55	**73.47**	67.77	72.57	75.76	73.40	76.30
CSL(YOLOv5-s) [83]	88.93	76.07	50.97	59.61	81.38	84.00	88.19	90.81	80.52	87.63	47.38	62.85	68.27	80.63	62.74	74.00
CSL(**EasyNAS(det)-s**)	88.64	84.78	51.02	62.04	81.30	84.02	88.23	90.82	85.79	87.36	52.20	65.68	73.72	74.33	65.93	75.72
CSL(YOLOv5-m) [83]	88.36	85.24	53.86	62.35	81.19	84.72	**88.57**	90.77	80.98	**88.09**	53.92	61.90	**75.99**	80.43	**68.59**	76.33
CSL(**EasyNAS(det)-m**)	89.04	86.61	53.83	63.05	**81.51**	**85.12**	88.46	90.77	**88.21**	87.93	56.04	60.96	75.96	**82.13**	66.18	**77.05**

R152 denotes ResNet-152 [1] and H-104 denotes hourglass-104 [74]. †: models are trained with multi-scale training techniques, while others are not. ⋆: two-stage detection framework, while others belong to a one-stage framework. Note CSL refers to the rotation detection loss which can be embodied by different backbones.

TABLE XII
EVALUATION OF SEMANTIC SEGMENTATION MODELS ON CITYSCAPE

Segmentation Method	mIoU (%)	FLOPs (G)	FPS	Search Cost (GPU-days)	Resolution
FCN-8S [40]	65.3	-	4.4	-	512x1024
PSPNet [85]	81.2	-	0.8	-	713x713
DeepLabV3 [41]†	81.3	-	1.3	-	769x769
Fast-SCNN [86]	68.0	-	123.5	-	1024x2048
AutoDeeplab [18]	80.3	695	-	3	1024x2048
CAS [87]	70.5	-	108	-	768x1536
GAS [88]	71.8	-	108	-	768x1536
SparseMask [89]	68.6	36.4	110	4.2	512x512
FasterSeg [17]	71.5	28.2	164	1.5	1024x2048
EasyNAS(seg)	72.6	28.2	173	0.7	1024x2048

Top: the performance of manually-designed architectures; Bottom: The performance of architectures searched by NAS. The search space is designed based on fasterseg [17] for fair comparison and differs from AutoDeeplab [18], as AutoDeeplab's codes for PyTorch platform are unavailable. †: extra coarse annotation data in cityscape in cityscape is utilized for training.

module. Our models are trained with rotation classification loss [36] from scratch for 300 epochs based on the open-sourced codes [82]. We also train YOLOv5 models on DOTA for comparison. Results are shown in Table XI . We observe that: *1)* Our EasyNAS(det)-YOLOv5s achieves better performance than many prior works in ResNet backbone; *2)* EasyNAS(det)-YOLOv5m outperforms the baseline (CSL [36] with ResNet-152 backbone) by 0.8% mAP_{50}; *3)* EasyNAS(det)-YOLOv5s model surpasses original YOLOv5-s model by more than 1.7% mAP_{50} and EasyNAS(det)-YOLOv5m surpasses YOLOv5-m by over 0.7%.

C. Architecture Search for Semantic Segmentation

1) Protocol: Semantic segmentation has been one of the most classic tasks in vision, which aims to allocate each pixel to pre-defined categories. We apply EasyNAS to the semantic segmentation task and derive EasyNAS(seg).

Dataset: Cityscape [89] is one of the most popular semantic segmentation datasets, containing 19 classes. Cityscape has 2975 training samples, 500 validation samples, and 1525 test samples. Each image is labeled per pixel.

Experimental settings: We refer to the settings in FasterSeg [17]. Specifically, the network weights are first pre-trained for 20 epochs with the architecture parameters fixed. Then we search the architecture by jointly optimizing the network weights and architecture parameters for another 30 epochs. The performance of the discovered architecture is obtained by training for 600 epochs from scratch.

2) Search Space (Standard): We refer to the search space of FasterSeg [17] and build the supernet based on our atomic search component. Fig. 8 shows the supernet architecture. We denote the feature map at layer l with s downsampling rate as z_l^s, which receives two inputs from z_{l-1}^s and $z_{l-1}^{s/2}$. Each connection has five candidate operations, including {1x1 conv, 3x3 conv, 5x5 conv, 3x3 dilated conv, skip connection} and three choices of reduction rate for output channel: {0.5, 0.75, 1.0}. Note that our search space is designed based on FasterSeg [17] and differs from AutoDeeplab [18], whose codes for PyTorch platform is unavailable.

3) Results on Cityscape: Table XII compares EasyNAS(seg) with other semantic segmentation methods. The first block reports the performance of manually-designed architectures, while the second block reports the performance of architecture searched by NAS methods. Specifically, we observe that manually-designed architectures have higher mIoU but low FPS, restricting their application on mobile devices and real-time scenarios. Though Fast SCNN [85] has sufficient operation speed, its accuracy is worse than the network we discover. In contrast, the searched architectures have both fast speed and

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

competitive mIoU performance. However, the prior NAS works usually require several GPU-days to search. In comparison, EasyNAS(seg) can discover an effective architecture in 0.7 GPU-day, outperforming other NAS-based methods, including FasterSeg and AutoDeeplab. Moreover, compared to FasterSeg, that have a similar search space to us, the architecture discovered by EasyNAS(seg) has higher (better) mIoU performance and faster speed, showing the good performance of our search method.

V. Conclusion

This paper has proposed a memory-efficient and fast architecture search method, dubbed EasyNAS, through an operator merging technique and dynamic channel refinement strategy. Compared to other sampling-based efficient NAS methods, EasyNAS maintains a static supernet structure during the search process and has a better convergence property, making it possible to directly search for complex computer vision tasks. Moreover, we propose an atomic search component that enables flexibly building supernets based on which a configurable search space-to-supernet conversion tool is devised for supporting custom search spaces for various tasks.

Acknowledgment

The authors are thankful to the anonymous reviewers for their valuable comments.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[2] T. Lin, P. Goyal, R. B. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020.

[3] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8697–8710.

[4] H. Pham, M. Y. Guan, B. Zoph, Q. V. Le, and J. Dean, "Efficient neural architecture search via parameter sharing," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4092–4101.

[5] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, and K. Kavukcuoglu, "Hierarchical representations for efficient architecture search," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–15.

[6] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4780–4789.

[7] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–15.

[8] X. Wang, C. Xue, J. Yan, X. Yang, Y. Hu, and K. Sun, "MergeNAS: Merge operations into one for differentiable architecture search," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 3065–3072.

[9] C. Xue, X. Wang, J. Yan, Y. Hu, X. Yang, and K. Sun, "Rethinking bi-level optimization in neural architecture search: A Gibbs sampling perspective," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 10551–10559.

[10] A. Zela, T. Elsken, T. Saikia, Y. Marrakchi, T. Brox, and F. Hutter, "Understanding and robustifying differentiable architecture search," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–15.

[11] X. Chen and C. Hsieh, "Stabilizing differentiable architecture search via perturbation-based regularization," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1554–1565.

[12] K. Bi, C. Hu, L. Xie, X. Chen, L. Wei, and Q. Tian, "Stabilizing darts with amended gradient estimation on architectural parameters," 2019, *arXiv:1910.11831*.

[13] H. Liang et al., "DARTS+: Improved differentiable architecture search with early stopping," 2019, *arXiv:1909.06035*.

[14] X. Chu, X. Wang, B. Zhang, S. Lu, X. Wei, and J. Yan, "DARTS-: Robustly stepping out of performance collapse without indicators," in *Proc. Int. Conf. Learn. Representations*, 2021.

[15] X. Wang, W. Guo, J. Su, X. Yang, and J. Yan, "ZARTS: On zero-order optimization for neural architecture search," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 1–15.

[16] Y. Chen, T. Yang, X. Zhang, G. Meng, X. Xiao, and J. Sun, "DetNAS: Backbone search for object detection," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 6638–6648.

[17] W. Chen, X. Gong, X. Liu, Q. Zhang, Y. Li, and Z. Wang, "FasterSeg: Searching for faster real-time semantic segmentation," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–15.

[18] C. Liu et al., "Auto-DeepLab: Hierarchical neural architecture search for semantic image segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 82–92.

[19] G. Bender, P. Kindermans, B. Zoph, V. Vasudevan, and Q. V. Le, "Understanding and simplifying one-shot architecture search," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 549–558.

[20] Y. Xu et al., "PC-DARTS: Partial channel connections for memory-efficient architecture search," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–15.

[21] X. Dong and Y. Yang, "Searching for a robust neural architecture in four GPU hours," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 1761–1770.

[22] C. Jiang, H. Xu, W. Zhang, X. Liang, and Z. Li, "SP-NAS: Serial-to-parallel backbone search for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11860–11869.

[23] G. Ghiasi, T.-Y. Lin, and Q. V. Le, "NAS-FPN: Learning scalable feature pyramid architecture for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 7036–7045.

[24] H. Xu, L. Yao, Z. Li, X. Liang, and W. Zhang, "Auto-FPN: Automatic network architecture adaptation for object detection beyond classification," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 6648–6657.

[25] N. Wang et al., "NAS-FCOS: Fast neural architecture search for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11940–11948.

[26] X. Wang, J. Lin, J. Zhao, X. Yang, and J. Yan, "EAutoDet: Efficient architecture search for object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 668–684.

[27] A. Wan et al., "FbNetV2: Differentiable neural architecture search for spatial and channel dimensions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12962–12971.

[28] X. Dong and Y. Yang, "NAS-BENCH-201: Extending the scope of reproducible neural architecture search," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–15.

[29] C. Liu et al., "Progressive neural architecture search," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 19–35.

[30] A. Bochkovskiy, C. Wang, and H. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.

[31] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "Scaled-YOLOV4: Scaling cross stage partial network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 13029–13038.

[32] J. Han, J. Ding, N. Xue, and G. Xia, "ReDet: A rotation-equivariant detector for aerial object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 2786–2795.

[33] X. Yang, J. Yan, M. Qi, W. Wang, Z. Xiaopeng, and T. Qi, "Rethinking rotated object detection with Gaussian Wasserstein distance loss," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 11830–11841.

[34] X. Yang, J. Yan, W. Liao, X. Yang, J. Tang, and T. He, "SCRDet++: Detecting small, cluttered and rotated objects via instance-level feature denoising and rotation loss smoothing," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 2384–2399, Feb. 2023.

[35] X. Yang, L. Hou, Y. Zhou, W. Wang, and J. Yan, "Dense label encoding for boundary discontinuity free rotation detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 15819–15829.

[36] X. Yang and J. Yan, "Arbitrary-oriented object detection with circular smooth label," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 677–694.

[37] L. Yao, H. Xu, W. Zhang, X. Liang, and Z. Li, "SM-NAS: Structural-to-modular neural architecture search for object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 12661–12668.

[38] J. Guo et al., "Hit-detector: Hierarchical trinity architecture search for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11402–11411.

[39] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 10778–10787.

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

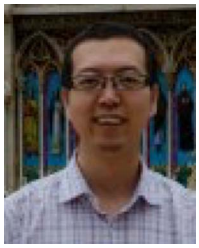
[40] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3431–3440.
[41] L. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," 2017, *arXiv:1706.05587*.
[42] Y.-C. Chen, C. Gao, E. Robb, and J.-B. Huang, "NAS-DIP: Learning deep image prior with neural architecture search," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 442–459.
[43] Y. Wang, M. Li, H. Cai, W.-M. Chen, and S. Han, "Lite pose: Efficient architecture design for 2D human pose estimation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 13126–13136.
[44] H. Tang et al., "Searching efficient 3D architectures with sparse point-voxel convolution," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 685–702.
[45] A. Mehrotra et al., "NAS-bench-ASR: Reproducible neural architecture search for speech recognition," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–15.
[46] Y. Du, T. Huang, S. You, M.-H. Hsieh, and D. Tao, "Quantum circuit architecture search for variational quantum algorithms," *npj Quantum Inf.*, vol. 8, no. 1, pp. 1–8, 2022.
[47] C. Xue, J. Yan, R. Yan, S. Chu, Y. Lin, and Y. Hu, "Transferable AutoML by model sharing over grouped datasets," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 9002–9011.
[48] Z. Lu, G. Sreekumar, E. D. Goodman, W. Banzhaf, K. Deb, and V. N. Boddeti, "Neural architecture transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 9, pp. 2971–2989, Sep. 2021.
[49] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–15.
[50] B. Moons et al., "Distilling optimal neural networks: Rapid search in diverse spaces," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 12209–12218.
[51] X. Wang et al., "ROME: Robustifying memory-efficient NAS via topology disentanglement and gradients accumulation," 2020, *arXiv:2011.11233*.
[52] X. Chen, L. Xie, J. Wu, and Q. Tian, "Progressive differentiable architecture search: Bridging the depth gap between search and evaluation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 1294–1303.
[53] G. Li, G. Qian, I. C. Delgadillo, M. Muller, A. Thabet, and B. Ghanem, "SGAS: Sequential greedy architecture search," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1617–1627.
[54] A. Noy et al., "ASAP: Architecture search, anneal and prune," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 493–503.
[55] J. Yu, L. Yang, N. Xu, J. Yang, and T. S. Huang, "Slimmable neural networks," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1554–1565.
[56] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style convnets great again," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 13733–13742.
[57] J. Mei et al., "AtomNAS: Fine-grained end-to-end neural architecture search," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–15.
[58] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," in *Handbook of Systemic Autoimmune Diseases*, 2009, vol. 1, no. 4.
[59] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
[60] S. Xie, H. Zheng, C. Liu, and L. Lin, "SNAS: Stochastic neural architecture search," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–15.
[61] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–15.
[62] H. Zhou, M. Yang, J. Wang, and W. Pan, "BayesNAS: A Bayesian approach for neural architecture search," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7603–7613.
[63] L. Li and A. Talwalkar, "Random search and reproducibility for neural architecture search," in *Proc. 35th Uncertainty Artif. Intell. Conf.*, A. Globerson and R. Silva, Eds., 2019, pp. 367–377.
[64] X. Dong and Y. Yang, "One-shot neural architecture search via self-evaluated template network," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 3680–3689.
[65] G. Jocher, "Yolov5 documentation," May 2020. [Online]. Available: https://docs.ultralytics.com/
[66] C. Wang, A. Bochkovskiy, and H. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," 2022, *arXiv:2207.02696*.
[67] X. Du et al., "SpineNet: Learning scale-permuted backbone for recognition and localization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11589–11598.
[68] S. Liu, D. Huang, and Y. Wang, "Learning spatial fusion for single-shot object detection," 2019, *arXiv:1911.09516*.
[69] T. Liang, Y. Wang, Z. Tang, G. Hu, and H. Ling, "OPANAS: One-shot path aggregation network architecture search for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 10195–10203.
[70] J. Redmon, "Darknet: Open source neural networks in c," 2013–2016. [Online]. Available: http://pjreddie.com/darknet/
[71] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.
[72] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 9626–9635.
[73] K. He, G. Gkioxari, P. Dollár, and R. B. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2980–2988.
[74] A. Newell, K. Yang, and J. Deng, "Stacked hourglass networks for human pose estimation," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 483–499.
[75] J. Wang, W. Yang, H. Li, H. Zhang, and G. Xia, "Learning center probability map for detecting objects in aerial images," *IEEE Trans. Geosci. Remote. Sens.*, vol. 59, no. 5, pp. 4307–4323, May 2021.
[76] H. Wei, Y. Zhang, Z. Chang, H. Li, H. Wang, and X. Sun, "Oriented objects as pairs of middle lines," *ISPRS J. Photogrammetry Remote Sens.*, vol. 169, pp. 268–279, 2020.
[77] Q. Ming, Z. Zhou, L. Miao, H. Zhang, and L. Li, "Dynamic anchor learning for arbitrary-oriented object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 2355–2363.
[78] L. Zhou, H. Wei, H. Li, W. Zhao, Y. Zhang, and Y. Zhang, "Arbitrary-oriented object detection in remote sensing images based on polar coordinates," *IEEE Access*, vol. 8, pp. 223373–223384, 2020.
[79] J. Yi, P. Wu, B. Liu, Q. Huang, H. Qu, and D. Metaxas, "Oriented object detection in aerial images with box boundary-aware vectors," in *Proc. Winter Conf. Appl. Comput. Vis.*, 2021, pp. 2150–2159.
[80] X. Pan et al., "Dynamic refinement network for oriented and densely packed object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11207–11216.
[81] P. Zhao, Z. Qu, Y. Bu, W. Tan, and Q. Guan, "PolarDet: A fast, more precise detector for rotated target in aerial images," *Int. J. Remote Sens.*, vol. 42, no. 15, pp. 5821–5851, 2021.
[82] H. Kaixuan, "Yolov5 for oriented object detection," 2020. [Online]. Available: https://github.com/hukaixuan19970627/yolov5_obb
[83] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 936–944.
[84] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 6230–6239.
[85] R. P. K. Poudel, S. Liwicki, and R. Cipolla, "Fast-SCNN: Fast semantic segmentation network," in *Proc. Brit. Mach. Vis. Conf.*, 2019, pp. 289–289.
[86] Y. Zhang, Z. Qiu, J. Liu, T. Yao, D. Liu, and T. Mei, "Customizable architecture search for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11641–11650.
[87] P. Lin, P. Sun, G. Cheng, S. Xie, X. Li, and J. Shi, "Graph-guided architecture search for real-time semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 4202–4211.
[88] H. Wu, J. Zhang, and K. Huang, "Sparsemask: Differentiable connectivity learning for dense image prediction," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 6767–6776.
[89] M. Cordts et al., "The cityscapes dataset for semantic urban scene understanding," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 3213–3223.

Xiaoxing Wang received the BE degree in electrical engineering from Shanghai Jiao Tong University, in 2018. He is currently working toward the PhD degree with the Department of Computer Science and Engineering, and AI Institute, Shanghai Jiao Tong University. He has published first-authored papers in IJCAI, ECCV, CVPR, NeurIPS on neural architecture search and once worked as research intern with IBM Research, Meituan, and JD.com. His main research interests include computer vision and machine learning.

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.

Zhirui Lian received the BS degree in applied mathematics from the School of Mathematics and Statistics, Xi'an Jiao Tong University, in 2022. He is currently working toward the master's degree majoring in applied statistics with the School of Mathematical Sciences, Shanghai Jiao Tong University. His main research interests include computer vision, deep learning, machine learning, and data mining.

Jiale Lin received the BE degree in software engineering from East China Normal University, and the master's degree in computer science from Shanghai Jiao Tong University. He is currently an algorithm engineer with Metabit Trading Technology, Shanghai, China. He once obtained the silver medal of ACM-CPC Qingdao (China) Regional Contest Silver Medal. His main research interests include machine learning and computer vision.



Chao Xue received the BS and MS degrees in information and telecommunication engineering from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2008 and 2011, respectively. He is currently working toward the PhD degree in computer science with BUPT and also a senior researcher with JD.com. His current research interests include hyper-parameter tuning, neural architecture search, AutoML, and large-scale machine learning.

Junchi Yan (Senior Member, IEEE) received the PhD degree in electrical engineering from Shanghai Jiao Tong University. He is currently an associate professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. Before that, he was a principal scientist with IBM Research – China, where he started his career in April 2011. He regularly serves as area chair for CVPR, AAAI, ICML, NeurIPS etc. and associate editor for *Pattern Recognition* and *ACM Trans. on Probabilistic Machine Learning*.

Authorized licensed use limited to: Instituto Nacional De Telecomunicações (INATEL). Downloaded on April 16,2026 at 11:04:05 UTC from IEEE Xplore. Restrictions apply.